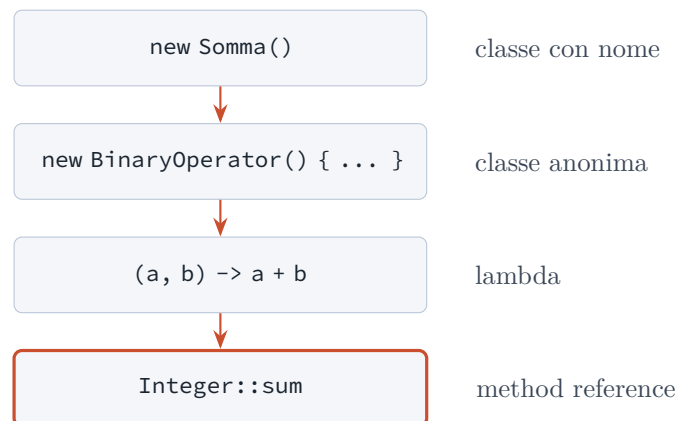


Dal nome alla funzione

Classi anonime, lambda expression,
method reference e interfacce funzionali



prof. Massimiliano Redolfi

Università degli Studi di Brescia · Ingegneria Informatica

massimiliano.redolfi@unibs.it

Versione 0.1

Quest'opera è distribuita con licenza [Creative Commons](https://creativecommons.org/licenses/by-nc-sa/4.0/)
"Attribuzione – Non commerciale – Condividi allo stesso
modo 4.0 Internazionale".



Indice

2	Dal nome alla funzione	1
2.1	Il problema: classi usa-e-getta	1
2.2	Classi anonime	2
2.3	Interfacce funzionali	4
2.4	Lambda expression	5
2.5	Method reference	7
2.6	A che cosa servono	9
2.7	Le interfacce di <code>java.util.function</code>	10
2.8	Il motore della calcolatrice, completo	11
2.9	Riepilogo	12
2.10	Esercizi	12
	Soluzioni	13

Dal nome alla funzione

Classi anonime, lambda expression, method reference e interfacce funzionali.

Il punto di partenza è un problema piccolo e concreto, il motore di una calcolatrice; il punto di arrivo è un modo diverso di pensare al codice: non più soltanto *oggetti* che hanno dei comportamenti, ma *comportamenti* che possono essere passati di mano come se fossero valori. Il percorso passa per tre costrutti del linguaggio, presentati nell'ordine in cui sono nati e in cui conviene impararli: la *classe anonima*, la *lambda expression* e il *method reference*. Alla fine vedremo che sono tre modi, sempre più compatti, di scrivere la stessa cosa, e che la cosa in questione è sempre un oggetto che implementa un'interfaccia con un solo metodo.

Le interfacce in Java moderno (metodi `default` e `static`, gerarchie `sealed`) sono l'argomento di una dispensa a parte: qui delle interfacce useremo solo ciò che serve alle lambda.

2.1 Il problema: classi usa-e-getta

Immaginiamo di scrivere una calcolatrice: deve applicare un operatore a due numeri. La struttura naturale, in un linguaggio a oggetti, è un'interfaccia che descrive *che cosa* fa un operatore, e una classe per ciascun operatore concreto che dice *come*:

```
public interface BinaryOperator {  
    int eval(int a, int b);  
}
```

```
public class Somma implements BinaryOperator {  
    public int eval(int a, int b) {  
        return a + b;  
    }  
}
```

Con Sottrazione e Potenza scritte allo stesso modo, il motore della calcolatrice diventa una tabella che associa il simbolo dell'operatore all'oggetto che lo calcola:

```
Map<String, BinaryOperator> operatori = new HashMap<>();  
operatori.put("+", new Somma());  
operatori.put("-", new Sottrazione());  
operatori.put("^", new Potenza());  
  
BinaryOperator op = operatori.get("+");  
System.out.println(op.eval(3, 4));    // 7
```

Funziona, ed è un buon progetto: aggiungere un operatore non richiede di toccare il motore. Ma guardiamo il costo. Per ogni operatore abbiamo scritto una classe, cioè un file, con un nome, un metodo e la cerimonia di `public class ... implements`. Di quella classe creiamo *una sola istanza*, nel momento stesso in cui la registriamo nella mappa, e non la useremo mai altrove. Il nome *Somma* non serve a nessuno: nessun altro pezzo di programma lo pronuncia.

È una situazione frequentissima. Ogni volta che una libreria ci chiede “un oggetto che sappia fare questa cosa” (confrontare due elementi per ordinarli, reagire al click di un bottone, eseguire un compito su un altro thread) ci troviamo a scrivere una classe che esiste per essere istanziata una volta. Il linguaggio ha sviluppato, in tre tappe, dei modi per scrivere *solo il comportamento*, lasciando al compilatore tutto il resto.

2.2 Classi anonime

Definizione 2.1: Classe anonima

Una *classe anonima* è una classe dichiarata nel punto stesso in cui se ne crea l'unica istanza, senza attribuirle un nome. La sintassi è quella di un'espressione `new` seguita dal corpo della classe:

`new Tipo(argomenti) { corpo della classe }`

dove *Tipo* è un'interfaccia da implementare o una classe da estendere.

Riscriviamo *Somma* come classe anonima:

```
BinaryOperator somma = new BinaryOperator() {  
    public int eval(int a, int b) {  
        return a + b;  
    }  
};
```

Leggiamola pezzo per pezzo, perché la sintassi mescola due cose che di solito stanno separate.

- `new BinaryOperator()` sembra la costruzione di un oggetto di tipo `BinaryOperator`. Non può esserlo, perché `BinaryOperator` è un'interfaccia. Il compilatore capisce che stiamo per *definire* una classe che la implementa. Le parentesi tonde contengono gli argomenti per il costruttore della classe padre; per un'interfaccia sono sempre vuote.
- Il blocco tra graffe è il *corpo della classe*: qui va tutto ciò che scriveremmo in *Somma*, in particolare il metodo `eval`.
- Il punto e virgola finale chiude l'istruzione di assegnamento: l'intera costruzione, dal `new` alla graffa chiusa, è un'espressione il cui valore è l'oggetto appena creato.

L'oggetto che ne risulta è un normale oggetto Java: ha un tipo, implementa `BinaryOperator`, può essere messo nella mappa esattamente come `new Somma()`. Ciò che è cambiato è solo *dove* sta scritta la classe.

Osservazione 2.1

Un nome, in realtà, la classe ce l'ha: il compilatore lo genera, e nel progetto compilato compare un file come `Calcolatrice$1.class`, cioè “la prima classe anonima dichiarata dentro `Calcolatrice`”. Lo rivedremo quando parleremo di reflection. Per noi il nome è irrilevante, ed è proprio questo il punto.

2.2.1 Che cosa può fare una classe anonima

Essendo una classe a tutti gli effetti, una classe anonima può dichiarare campi e metodi propri, oltre a quelli richiesti dall'interfaccia, e può avere un blocco di inizializzazione. Soprattutto, può usare ciò che le sta attorno: i campi e i metodi della classe in cui è dichiarata, e le variabili locali del metodo in cui compare. Questa seconda capacità è la più importante, e merita una sezione a sé (§2.2.2).

Ci sono anche cose che non può fare, tutte conseguenza del non avere un nome:

- non può dichiarare un *costruttore*: un costruttore si chiama come la classe. Se serve inizializzare qualcosa, si usa un blocco di inizializzazione `{ ... }` nel corpo;
- non può essere istanziata una *seconda volta*: non c'è un nome con cui richiamarla. Se servono due istanze, serve una classe con nome;
- non può implementare *più di un'interfaccia*, né estendere una classe e insieme implementare un'interfaccia: dopo `new` c'è posto per un solo tipo.

2.2.2 Accesso al contesto e variabili catturate

Il vero vantaggio di dichiarare una classe *dentro* un metodo è che la classe vede ciò che il metodo vede. Consideriamo un metodo che costruisce un operatore parametrizzato:

```
static BinaryOperator sommaScalata(int k) {  
    return new BinaryOperator() {  
        public int eval(int a, int b) {  
            return k * (a + b);    // k e' il parametro del metodo esterno  
        }  
    };  
}
```

```
BinaryOperator doppia = sommaScalata(2);  
System.out.println(doppia.eval(3, 4));    // 14
```

C'è qualcosa di sorprendente. Quando `doppia.eval(3, 4)` viene eseguito, il metodo `sommaScalata` è terminato da un pezzo: il suo parametro `k`, che viveva nello stack di quella chiamata, non esiste più. Eppure `eval` lo usa e trova il valore giusto.

Il meccanismo è semplice: al momento della creazione dell'oggetto anonimo, il compilatore *copia* il valore di `k` in un campo nascosto dell'oggetto. L'oggetto porta con sé una fotografia del contesto in cui è nato. Un blocco di codice insieme alle variabili del contesto che usa si chiama *closure* (chiusura), ed è un concetto che ritroverete in ogni linguaggio moderno.

Definizione 2.2: Variabile effectively final

Una variabile locale (o un parametro) è *effectively final* se, dopo l'inizializzazione, non viene mai riassegnata: se cioè si potrebbe aggiungere la parola `final` alla sua dichiarazione senza che il programma smetta di compilare.

La copia spiega una regola che altrimenti sembra arbitraria: una classe anonima (e, come vedremo, una lambda) può usare solo variabili locali *effectively final*. Se la variabile potesse cambiare dopo la copia, esisterebbero due valori diversi con lo stesso nome, uno nel metodo e uno nell'oggetto, e il programma diventerebbe imprevedibile. Il compilatore preferisce vietare.

Attenzione 2.1

Il codice seguente non compila:

```
int contatore = 0;
Runnable r = new Runnable() {
    public void run() { contatore++; } // errore: non effectively final
};
```

L'errore non è nel `++` in sé: è che `contatore` viene modificata, quindi non è più una costante che si possa fotografare. Se davvero serve uno stato che cambia, va messo in un oggetto (un campo della classe, o un oggetto contenitore), non in una variabile locale.

Osservazione 2.2

Dentro una classe anonima, `this` è l'oggetto anonimo, non l'oggetto della classe che la contiene. Per raggiungere quest'ultimo si scrive `NomeClasseEsterna.this`. È una delle differenze rispetto alle lambda, dove `this` conserva il significato che ha nel metodo circostante.

2.3 Interfacce funzionali

Prima di passare alle lambda dobbiamo dare un nome a una proprietà che `BinaryOperator` ha e che finora abbiamo usato senza dirlo: ha *un solo* metodo astratto.

Definizione 2.3: Interfaccia funzionale

Un'*interfaccia funzionale* è un'interfaccia con esattamente un metodo astratto. Non contano, ai fini del conteggio, i metodi `default` e `static` (che hanno un corpo), né i metodi di `Object` eventualmente ridichiarati (come `equals`).

Sono funzionali `Runnable` (un solo metodo, `run`), `Comparator` (il metodo astratto è `compare`; gli altri, e sono decine, sono `default` o `static`) e la nostra `BinaryOperator`. Non lo è `List`, che ne ha molti.

Perché questa proprietà conta? Perché se un'interfaccia ha un solo metodo astratto, *implementarla* significa una cosa sola: scrivere il corpo di quel metodo. Il nome del metodo, il suo tipo di ritorno e i tipi dei parametri sono già noti dall'interfaccia. Tutto ciò che nell'implementazione non è il corpo è, a rigore, ridondante.

Java 8 ha introdotto un'annotazione per dichiarare questa intenzione:

```
@FunctionalInterface
public interface BinaryOperator {
    int eval(int a, int b);
}
```

L'annotazione non cambia il comportamento dell'interfaccia. Serve a due cose: documenta, per chi legge, che l'interfaccia è pensata per essere implementata con una lambda; e chiede al compilatore di *rifiutare* l'interfaccia se qualcuno, in futuro, aggiungesse un secondo metodo astratto rompendo tutte le lambda esistenti. Conviene usarla sulle proprie interfacce funzionali.

2.4 Lambda expression

Riprendiamo la classe anonima di pagina 2 e chiediamoci che cosa, in quelle cinque righe, è davvero informazione e che cosa è ripetizione.

```
BinaryOperator somma = new BinaryOperator() {
    public int eval(int a, int b) {
        return a + b;
    }
};
```

Il tipo `BinaryOperator` compare due volte. Il nome del metodo, `eval`, è obbligato: è l'unico metodo astratto. I tipi dei parametri, `int` e `int`, e il tipo di ritorno sono scritti nell'interfaccia. Ciò che resta di nostro sono i *nomi dei parametri* e il *corpo*. Una lambda expression è esattamente questo.

Definizione 2.4: Lambda expression

Una *lambda expression* è un'espressione della forma

$$(parametri) \rightarrow corpo$$

il cui valore è un oggetto che implementa un'interfaccia funzionale. Il corpo è un'espressione, il cui valore è il risultato, oppure un blocco tra graffe con istruzioni e, se il metodo restituisce qualcosa, un `return` esplicito.

La classe anonima diventa, in due passi successivi:

```
BinaryOperator somma = (a, b) -> { return a + b; };
```

```
BinaryOperator somma = (a, b) -> a + b;
```

Nella prima forma il corpo è un blocco, e il `return` è esplicito. Nella seconda il corpo è una sola espressione, e il suo valore è automaticamente il risultato. La seconda forma è quella che si usa quando possibile: è la più leggibile.

2.4.1 La sintassi, per esteso

Le varianti sono poche e tutte prevedibili:

```
( ) -> 42 // nessun parametro: parentesi obbligatorie
x -> x * 2 // un parametro: parentesi facoltative
(x, y) -> x + y // due o piu' parametri
(int x, int y) -> x + y // tipi espliciti, se l'inferenza non basta
x -> { log(x); return x * 2; } // blocco: piu' istruzioni, return esplicito
( ) -> { System.out.println("ciao"); } // blocco senza valore di ritorno
```

I tipi dei parametri normalmente si omettono: il compilatore li *deduce* dall'interfaccia che la lambda deve implementare. Se se ne dichiara uno, vanno dichiarati tutti.

2.4.2 Il tipo di una lambda

C'è una domanda che la sintassi nasconde: di che tipo è $(a, b) \rightarrow a + b$? La risposta è che, da sola, *non ha* un tipo. La stessa scrittura può implementare la nostra `BinaryOperator`, oppure `IntBinaryOperator` della libreria standard, oppure qualunque altra interfaccia funzionale con due parametri numerici e un risultato numerico. È il *contesto* a decidere: il tipo della variabile a cui la lambda è assegnata, o il tipo del parametro del metodo a cui è passata. Questo meccanismo si chiama *target typing*.

```
BinaryOperator s1 = (a, b) -> a + b; // metodo implementato: eval
IntBinaryOperator s2 = (a, b) -> a + b; // metodo implementato: applyAsInt
```

Due conseguenze pratiche. La prima: una lambda si può scrivere solo dove il compilatore conosce il tipo atteso; `var f = (a, b) -> a + b;` non compila, perché `var` chiede al compilatore di dedurre il tipo dall'espressione, e l'espressione non ne ha uno. La seconda: l'interfaccia bersaglio deve essere funzionale. Se ha due metodi astratti, il compilatore non sa quale dei due la lambda stia implementando, e rifiuta.

2.4.3 Che cosa cambia rispetto alla classe anonima

Una lambda non è semplicemente “una classe anonima scritta corta”. Le regole sulle variabili catturate sono le stesse (§2.2.2: solo variabili *effectively final*), ma ci sono due differenze che vale la pena fissare.

- **this.** Dentro una lambda, `this` è l'oggetto in cui la lambda è scritta, esattamente come nel codice circostante. La lambda non introduce un nuovo `this`. Dentro una classe anonima, invece, `this` è l'oggetto anonimo.
- **Nessun nuovo ambito di nomi.** Una lambda non può dichiarare un parametro o una variabile locale con lo stesso nome di una variabile del metodo che la contiene: sarebbe un errore di ridefinizione, come per qualunque blocco. Una classe anonima, essendo una classe, può ombreggiare i nomi esterni.

E una differenza di uso: la lambda è fatta per i comportamenti *brevi*. Se il corpo supera la manciata di righe, se servono campi per tenere uno stato, se l'interfaccia ha più metodi, la classe anonima (o una classe con nome) resta lo strumento giusto.

2.5 Method reference

Molte lambda hanno un corpo che si limita a chiamare un metodo già esistente, passando gli stessi parametri che ricevono:

```
Consumer<String> stampa = s -> System.out.println(s);
```

Qui la lambda non aggiunge niente: prende `s` e lo passa a `println`. Il linguaggio offre una scrittura ancora più diretta, che nomina il metodo e basta.

Definizione 2.5: Method reference

Un *method reference* è un'espressione della forma *qualificatore::nome*, che denota un metodo (o un costruttore) esistente. Come una lambda, il suo valore è un oggetto che implementa l'interfaccia funzionale attesa dal contesto; i parametri del metodo astratto vengono passati, in ordine, al metodo referenziato.

```
Consumer<String> stampa = System.out::println;
```

Le forme sono quattro, e si distinguono da ciò che sta a sinistra dei due punti.

Forma	Esempio	Lambda equivalente
oggetto::metodo	System.out::println	<code>x -> System.out.println(x)</code>
Classe::metodoStatico	Math::sqrt	<code>x -> Math.sqrt(x)</code>
Classe::metodoDiIstanza	String::compareTo	<code>(s, t) -> s.compareTo(t)</code>
Classe::new	ArrayList::new	<code>() -> new ArrayList<>()</code>

Le prime due forme sono trasparenti: il metodo riceve i parametri della lambda così come sono. La terza merita attenzione. `String::compareTo` nomina un metodo *di istanza* senza dire su quale istanza chiamarlo: la regola è che il *primo* parametro della lambda diventa l'oggetto su cui si chiama il metodo, e gli altri diventano gli argomenti. Per un `Comparator<String>`, che riceve due stringhe, questo significa `(s, t) -> s.compareTo(t)`: esattamente il confronto che serve per ordinare. La quarta forma referencia un costruttore, e si usa quando un'interfaccia chiede “un modo per creare un oggetto” (Supplier, §2.7).

Esempio 2.1: Ordinare una lista di nomi

List ha un metodo sort che riceve un Comparator. Le tre scritture seguenti sono equivalenti e ordinano alfabeticamente:

```
List<String> nomi = new ArrayList<>(List.of("Rossi", "Bianchi", "Verdi"));

nomi.sort(new Comparator<String>() {
    public int compare(String s, String t) { return s.compareTo(t); }
});
nomi.sort((s, t) -> s.compareTo(t));
nomi.sort(String::compareTo);
```

Per ordinare per lunghezza, invece, serve una lambda vera, perché il confronto non coincide con un metodo esistente:

```
nomi.sort((s, t) -> Integer.compare(s.length(), t.length()));
```

2.5.1 La scala completa

Possiamo ora mettere in fila le quattro scritture dello stesso operatore:

```
operatori.put("+", new Somma()); // 1. classe con nome

operatori.put("+", new BinaryOperator() { // 2. classe anonima
    public int eval(int a, int b) { return a + b; }
});

operatori.put("+", (a, b) -> a + b); // 3. lambda

operatori.put("+", Integer::sum); // 4. method reference
```

La riga 4 usa `Integer.sum(int, int)`, un metodo statico della libreria che fa la somma: la sua firma coincide con quella di `eval`, quindi il method reference è lecito. Tutte e quattro le righe mettono nella mappa un oggetto che implementa `BinaryOperator`; il motore della calcolatrice, che chiama `eval`, non si accorge di quale forma sia stata usata. Cambia solo la quantità di rumore attorno al comportamento, cioè attorno ad `a + b`.

Ricetta: scegliere la forma

1. **Il corpo chiama un solo metodo esistente, con gli stessi parametri?** Method reference.
2. **Il corpo è un'espressione o poche istruzioni?** Lambda, nella forma senza graffe se possibile.
3. **Serve uno stato (campi), o l'interfaccia ha più metodi, o il corpo è lungo?** Classe anonima.
4. **Serve istanziarla più volte, o riusarla altrove?** Classe con nome.

2.6 A che cosa servono

Finora le lambda ci hanno fatto risparmiare righe. Il loro vero valore è un altro, e per vederlo partiamo da due metodi quasi identici:

```
static void interi(int n) {  
    for (int i = 0; i < n; i++)  
        System.out.println(i);  
}
```

```
static void pari(int n) {  
    for (int i = 0; i < n; i++)  
        if (i % 2 == 0) System.out.println("pari: " + i);  
}
```

Il ciclo è lo stesso. Cambia *che cosa si fa* a ogni passo. In un linguaggio senza lambda non c'è modo di scrivere il ciclo una volta sola e di passargli “la cosa da fare” come parametro: un parametro deve essere un valore, e un pezzo di codice non lo è. Con le interfacce funzionali lo diventa.

```
static void ripeti(int n, IntConsumer azione) {  
    for (int i = 0; i < n; i++)  
        azione.accept(i);  
}
```

`IntConsumer` è un'interfaccia funzionale della libreria standard con un solo metodo, `void accept(int)`: “ricevi un intero e fanne qualcosa”. Ora i due metodi di prima sono due *chiamate*:

```
ripeti(5, i -> System.out.println(i));  
ripeti(5, i -> { if (i % 2 == 0) System.out.println("pari: " + i); });
```

Quando l'indice non serve, l'interfaccia giusta è `Runnable`, il cui metodo `run` non riceve nulla:

```
static void ripeti(int n, Runnable azione) {  
    for (int i = 0; i < n; i++)  
        azione.run();  
}  
  
ripeti(3, () -> System.out.println("Hello!"));  
ripeti(3, () -> System.out.print("x"));
```

Abbiamo *iniettato* un comportamento in un metodo. Il metodo `ripeti` è scritto una volta e non sa che cosa farà; chi lo chiama decide. Questa inversione, il codice che riceve codice, è lo schema di base di quasi tutte le API moderne, e la incontreremo in cinque situazioni ricorrenti:

- **eseguire lo stesso codice in punti diversi**, come `ripeti`;
- **iniettare logica in metodi di terzi**: `list.sort(...)` riceve il criterio di ordinamento, `map.computeIfAbsent(...)` riceve come costruire il valore mancante, gli stream ricevono filtri e trasformazioni;

- **reagire a un evento:** `button.addActionListener(e -> ...)` registra che cosa fare al click; il codice verrà eseguito chissà quando, e chissà quante volte;
- **eseguire codice solo se serve:** un logger che riceve `() -> messaggioCostoso()` lo valuta solo se il livello di log lo richiede;
- **eseguire codice su un altro thread:** `new Thread(() -> ...)` riceve il lavoro da svolgere altrove.

In tutti i casi la lambda dichiara un blocco di codice *qui* perché venga eseguito *altrove*, *dopo*, forse *più volte*. Il “dopo” è il motivo per cui le variabili catturate devono essere costanti: quando il codice girerà, il contesto che le conteneva potrebbe non esistere più (§2.2.2).

2.7 Le interfacce di `java.util.function`

Non serve dichiarare un'interfaccia funzionale per ogni forma di comportamento: le forme ricorrenti sono poche, e la libreria standard le fornisce nel package `java.util.function`. Conviene impararle per nome, perché le si incontra in tutta l'API.

Interfaccia	Metodo	Firma	Esempio
<code>Supplier<T></code>	<code>get</code>	<code>() -> T</code>	<code>() -> new Random()</code>
<code>Consumer<T></code>	<code>accept</code>	<code>T -> void</code>	<code>s -> System.out.println(s)</code>
<code>Function<T,R></code>	<code>apply</code>	<code>T -> R</code>	<code>s -> s.length()</code>
<code>Predicate<T></code>	<code>test</code>	<code>T -> boolean</code>	<code>s -> s.isEmpty()</code>
<code>UnaryOperator<T></code>	<code>apply</code>	<code>T -> T</code>	<code>s -> s.toUpperCase()</code>
<code>BinaryOperator<T></code>	<code>apply</code>	<code>(T, T) -> T</code>	<code>(a, b) -> a + b</code>
<code>BiFunction<T,U,R></code>	<code>apply</code>	<code>(T, U) -> R</code>	<code>(s, n) -> s.repeat(n)</code>

Il nome dice il ruolo: un *supplier* fornisce un valore senza riceverne, un *consumer* ne riceve uno senza restituire nulla, una *function* trasforma, un *predicate* risponde sì o no, un *operator* è una funzione che resta nello stesso tipo. `Runnable`, che non riceve né restituisce, sta in `java.lang` perché esiste da prima delle lambda.

Per i tipi primitivi esistono varianti dedicate, che evitano il costo del *boxing* (la conversione tra `int` e `Integer`): `IntConsumer`, `IntPredicate`, `IntFunction<R>`, `ToIntFunction<T>`, `IntBinaryOperator`, e le omologhe per `long` e `double`. La nostra `BinaryOperator` con `int eval(int, int)` è, a meno del nome del metodo, `IntBinaryOperator`: in un progetto reale useremmo direttamente quella.

Attenzione 2.2

Il nome `BinaryOperator` è usato sia dalla nostra interfaccia sia da `java.util.function.BinaryOperator<T>`. Se in un file si importa `java.util.function.*`, le due entrano in conflitto e il compilatore chiede di qualificare il nome. Qui il nome è tenuto perché dice esattamente che cosa fa l'interfaccia; in un progetto che usa entrambe conviene rinominare la propria (per esempio `Operazione`), o usare direttamente `IntBinaryOperator`.

2.7.1 Comporre

Queste interfacce hanno anche metodi `default` che permettono di combinare comportamenti senza scrivere nuove lambda. I più utili:

```
Predicate<String> vuota    = String::isEmpty;
Predicate<String> nonVuota = vuota.negate();
Predicate<String> corta   = s -> s.length() < 4;
Predicate<String> ok      = nonVuota.and(corta);           // non vuota E corta

Function<String, Integer> lunghezza = String::length;
Function<String, Integer> doppia    = lunghezza.andThen(n -> n * 2);
System.out.println(doppia.apply("ciao"));                 // 8
```

`andThen` esegue prima la funzione su cui è chiamato e poi l'argomento; `compose` fa il contrario. Non serve impararli a memoria: serve sapere che esistono, per cercarli nella documentazione quando ci si trova a scrivere una lambda che ne richiama due.

2.8 Il motore della calcolatrice, completo

Chiudiamo il cerchio con il problema di partenza. Con le lambda il motore sta in una schermata:

```
public class Motore {
    private final Map<String, BinaryOperator> operatori = new HashMap<>();

    public Motore() {
        operatori.put("+", (a, b) -> a + b);
        operatori.put("-", (a, b) -> a - b);
        operatori.put("*", (a, b) -> a * b);
        operatori.put("^", (a, b) -> (int) Math.pow(a, b));
    }

    public int calcola(String op, int a, int b) {
        BinaryOperator f = operatori.get(op);
        if (f == null)
            throw new IllegalArgumentException("operatore sconosciuto: " + op);
        return f.eval(a, b);
    }
}
```

Rispetto alla versione con una classe per operatore non è cambiata l'architettura: c'è ancora un'interfaccia, ci sono ancora oggetti che la implementano, la mappa fa ancora da tabella di dispatch. È cambiato il rapporto tra ciò che scriviamo e ciò che conta: quattro righe, una per operatore, ciascuna con la sola formula. Aggiungere il modulo è una riga.

```
Motore m = new Motore();
System.out.println(m.calcola("^", 2, 10)); // 1024
System.out.println(m.calcola("%", 7, 3));  // IllegalArgumentException
```

2.9 Riepilogo

- Una **classe anonima** è una classe dichiarata dove se ne crea l'unica istanza: `new Tipo() { ... }`. Vede il contesto in cui è scritta e ne *cattura* le variabili locali, che devono essere *effectively final*.
- Un' **interfaccia funzionale** ha un solo metodo astratto; `@FunctionalInterface` lo dichiara e lo fa verificare.
- Una **lambda** è (parametri) -> corpo: l'implementazione di un'interfaccia funzionale ridotta a parametri e corpo. Non ha un tipo proprio: lo riceve dal contesto (*target typing*). `this` è quello del codice circostante.
- Un **method reference** `qualificatore::metodo` è una lambda che si limita a chiamare un metodo esistente; nella forma `Classe::metodoDiIstanza` il primo parametro diventa l'oggetto ricevente.
- Le lambda servono a **iniettare comportamento**: un metodo riceve come parametro che cosa fare, e lo esegue altrove, dopo, forse più volte.
- `java.util.function` fornisce le interfacce ricorrenti: `Supplier`, `Consumer`, `Function`, `Predicate`, gli `Operator` e le varianti per i primitivi.

2.10 Esercizi

Esercizio 2.1: Da classe anonima a lambda

Riscrivere ciascuna delle classi anonime seguenti come lambda e, dove possibile, come method reference.

```
Runnable r = new Runnable() {  
    public void run() { System.out.println("tick"); }  
};  
Comparator<Integer> c = new Comparator<Integer>() {  
    public int compare(Integer a, Integer b) { return Integer.compare(a,  
        b); }  
};  
Function<String, String> f = new Function<String, String>() {  
    public String apply(String s) { return s.trim(); }  
};
```

Esercizio 2.2: Funzionale o no?

Per ciascuna interfaccia dire se è funzionale e, in caso affermativo, scrivere una lambda che la implementi.

```
interface A { void f(); }  
interface B { void f(); void g(); }  
interface C { void f(); default void g() { f(); f(); } }  
interface D { boolean equals(Object o); int h(int x); }  
interface E { }
```

Esercizio 2.3: Il contatore che non compila

Si vuole contare quante volte `ripeti` esegue l'azione:

```
int conteggio = 0;
ripeti(5, i -> conteggio++);
System.out.println(conteggio);
```

Spiegare perché non compila e proporre due modi diversi di ottenere il conteggio.

Esercizio 2.4: Una propria interfaccia funzionale

Dichiarare un'interfaccia funzionale `Trasformazione` con un metodo `String applica(String s)`, e un metodo statico `String applicaTutte(String s, List<Trasformazione> ts)` che applica le trasformazioni in sequenza. Usarlo con tre lambda: rimozione degli spazi ai bordi, conversione in maiuscolo, aggiunta di un punto esclamativo finale.

Esercizio 2.5: Il motore, esteso

Aggiungere al motore della §2.8 gli operatori `%` (resto), `max` e `min`, usando per questi ultimi due un method reference. Poi aggiungere un metodo `Set<String> operatoriDisponibili()` utile al comando `help` della calcolatrice.

Soluzioni

Soluzione 2.1.

```
Runnable r = () -> System.out.println("tick");
Comparator<Integer> c = (a, b) -> Integer.compare(a, b);
Function<String, String> f = s -> s.trim();
```

Le ultime due si possono scrivere anche `Integer::compare` e `String::trim`. `Integer::compare` è un metodo statico che riceve i due parametri così come sono (seconda forma della tabella); `String::trim` è un metodo di istanza, e il parametro `s` diventa l'oggetto su cui viene chiamato (terza forma). Per `Runnable` non c'è method reference: `System.out::println` senza argomenti stamperebbe una riga vuota, che è un'altra cosa.

Soluzione 2.2. A è funzionale: `A a = () -> {};`. B non lo è: due metodi astratti. C è funzionale: `g` ha un corpo (default) e non conta; `C c = () -> System.out.println("f");`. D è funzionale: `equals` è un metodo di `Object` e non conta, resta `h`; `D d = x -> x + 1;`. E non è funzionale: zero metodi astratti, e una lambda non saprebbe che cosa implementare.

Soluzione 2.3. La lambda cattura conteggio, che è una variabile locale; `conteggio++` la modifica, quindi non è *effectively final* e il compilatore rifiuta la cattura. Due soluzioni. La prima è dare al contatore un contenitore che non cambia, mentre cambia il suo contenuto: un array di un elemento, `int[] conteggio = {0};` con `conteggio[0]++` nella lambda (il riferimento all'array è costante), oppure la classe `AtomicInteger` della libreria, pensata proprio per questo. La seconda, spesso migliore, è non contare per effetto collaterale: se `ripeti` esegue l'azione esattamente `n` volte, il conteggio è `n` e non c'è nulla da misurare; altrimenti si ristrutturava il codice perché sia il metodo, non la lambda, a restituire il numero di esecuzioni.

Soluzione 2.4.

```
@FunctionalInterface
interface Trasformazione {
    String applica(String s);
}

static String applicaTutte(String s, List<Trasformazione> ts) {
    for (Trasformazione t : ts)
        s = t.applica(s);
    return s;
}

String r = applicaTutte(" ciao mondo ",
    List.of(String::trim, String::toUpperCase, s -> s + "!"));
System.out.println(r);    // CIAO MONDO!
```

Le prime due trasformazioni sono method reference della terza forma; la terza è una lambda perché aggiunge qualcosa che nessun metodo esistente fa da solo. Notare che `s = t.applica(s)` riassegna il *parametro* `s` del metodo, e non una variabile catturata: è lecito.

Soluzione 2.5. Nel costruttore:

```
operatori.put("%", (a, b) -> a % b);
operatori.put("max", Math::max);
operatori.put("min", Math::min);
```

`Math.max(int, int)` e `Math.min(int, int)` hanno la stessa firma di `eval`, quindi il method reference è lecito. Il metodo richiesto restituisce le chiavi della mappa: `return operatori.keySet();`. Restituire direttamente il `keySet` espone la mappa a modifiche dall'esterno; se si vuole evitarlo, `return Set.copyOf(operatori.keySet());`.